



CS 3110 – Software Engineering

Access Control System (Final Project)

LAKB.AI – Intelligent Travel Assistant Application

GROUP: LOCaIT

SECTION: BSCS - 3A

GROUP MEMBERS: AQUINO, SEAN XANDER B.
ATIENZA, LAWRENCE
ORIAS, MARK JOSEPH C.

1. Introduction

1.1. Summary

LAKB.AI is an *AI-powered travel recommendation system* designed to help users generate trip plans, discover destinations, and retrieve real-time information using geolocation, Google Places API, and AI recommendations. The system includes offline caching, secure API access handling, and role-based access to administrative features.

The goal of LAKB.AI is to provide **fast, contextual, and intelligent trip planning** while maintaining strong privacy, access control, and data protection practices.

1.2. Purpose

This document defines the functional and non-functional requirements for the Lakb.ai, version 1.0. Its purpose is to provide a clear and consistent guide for the development team and to serve as the primary point of reference for project stakeholders regarding the system's intended behavior and capabilities.

By clearly defining the requirements, it ensures seamless collaboration among the development team throughout the build process. It helps reduce misunderstandings and mishaps and ensures that the system's development is aligned with its intended purpose.

This document ensures that:

- Developers understand what to build
- Designers understand what to visualize
- Testers and Documentators understand what to validate



1.3. Scope

Lakb.ai is a mobile-first AI travel assistant that serves:

a. For Travelers & Tourists

- Discover new destinations and activities based on preferences
- Generate personalized multi-day itineraries
- Access real-time weather, holidays, and air quality information
- Save favorite places and sync across devices

b. For Local Residents

- Explore hidden gems and local attractions
- Find nearby points of interest
- Discover new experiences in familiar areas

c. For Travel Agencies

- Promotes tourism by showcasing popular and emerging local destinations

2. System Description

2.1. Product Perspective

Lakb.ai is a **standalone mobile application** that integrates:

- AI recommendation engine
- Google Places API (data, images, reviews, routing)
- Geolocation services

It uses user preferences + mood + real-time location to compute recommendations and itineraries.

2.2. Product Function

Core system functions:

- Location-based Point-of-Interest (POI) search
- Preference filtering (budget, activity type, travel style)
- Destination details with photos & metadata
- AI-generated itineraries



- Routing & maps
- User accounts / saved favorites
- Review aggregation (Google Place API)

2.3. Frameworks / Technologies Used & Rationale

Flet (Python UI Framework)

Flet was out of choice given its centralized strength of creating platform-independent, visually appealing user interfaces with a single Python implementation. This decision drastically reduced the development time, removed the necessity to have independent native mobile and web development teams and enabled us to use our existing Python experience to write the backend logic as well as the frontend presentation.

AI / Generative Model (Gemini API)

The Gemini API is selected to be the central intelligence of LAKB.AI. Its sophisticated logic and formatted output features play a vital role in processing sophisticated, open-ended travel bookings and creating precise and tailored itineraries in a foreseeable, JSON form. Conversational planning is made possible through this model and it offers a rich user experience, as opposed to plain database lookups.

Google Places API / Maps / OpenWeather / OpenAQ / Calendarific

External APIs deliver real time context sensitive information. Google Places API is necessary to get correct location information, points of interest as well as navigational context. It includes OpenWeather to get instant weather predictions so that, as per the requirements, the AI could adjust clothing or activity suggestions, thus, being a significant component of a complete travel assistant.

Local Caching (JSON Fallback)

The application uses minimal local caching through JSON files and Flet's client storage for offline fallback scenarios:

- **Recent Searches:** Stored in Flet's `client_storage` for quick access
- **Favorites Fallback:** Cached locally in `favorites.json` when offline
- **Auth Tokens:** Stored in `client_storage` for session persistence
- **User ID Caching:** In-memory caching to reduce redundant API calls



This caching strategy is minimal by design - large datasets like place information, maps, and full itineraries are NOT cached offline. The application prioritizes cloud synchronization over local storage.

SUPABASE (Primary Database)

Supabase (PostgreSQL) is the primary cloud database and authentication provider for the application:

- **Authentication:** Email/password and Google OAuth flows via Supabase Auth
- **Database Tables:** Three main tables with Row Level Security (RLS):
 - **profiles** - Extended user information (name, avatar, etc.)
 - **favorites** - User-saved places with JSONB data
 - **plans** - Travel itineraries with JSONB data
- **Storage:** Profile avatars stored in Supabase Storage **avatars** bucket
- **Real-time Sync:** Cross-device data synchronization
- **Security:** Row Level Security ensures users only access their own data

2.4. Architecture & Module Overview

A clear and modular architecture of LAKB.AI allows maintaining, scaling, and separation of concerns. The application consists of the following major modules that interact mostly via the API Service Module:

Authentication Module

This module is user session management in case it is implemented. It does the email/password, or Google Sign-In, login processing, state of the session, and user registration and profile changes.

AI Engine Module

This is the main intelligence element of the application. Its main role is to process complicated queries of the user, and create multi-day context aware itineraries. It accepts user preferences, location, and merges API Service Module (weather, holidays, air quality) data to generate a structured JavaScript Object (JSON).

- Generates itineraries
- Accepts user preferences and location
- It combines data from the API Service Module (weather, holidays, air quality) to produce a structured JavaScript Object (JSON).
- Produces trip summaries



- Combines weather + holiday + air quality context

API Service Module

This is the point of convergence of all external communications. It can handle authorized requests to all third-party services leaving no API keys on the client-side.

- Google Places Search: Processes search results, place details, images downloading, and map URL generation.
- Place Details
- Maps URL generation
- AI (Gemini) requests
- Weather, Air Quality, Holidays

State Management

Custom state controllers manage application state and data flow:

- **auth_state_controller** - Authentication session management
- **places_state_controller** - Place search results and caching
- **favorites_state_controller** - User favorites synchronization
- **profile_state_controller** - User profile data
- **app_state_manager** - Global application state
- **service_manager** - Service dependency injection

UI / Views Module

This is the presentation layer that is constructed with Flet framework. It is in charge of the visualization of any visual content and processing of user input. It is the main application views:

- Home: This is the first area of quick search and customized content.
- Search Results
- Destination Page: Full information of a chosen POI.
- Plan Builder: This is where the user enters his/her preferences and is shown the completed itinerary.
- Generated Plan Summary

2.5. Implemented Features

Baseline Features

- AI-powered trip generation



- Realtime place search with images
- Limited offline support (JSON fallback for favorites and auth tokens)
- My Plans (save/load trip plans)
- Destination details page
- Geolocation tracking
- Map preview

Additional Enhancements

- Travel tips generation
- Smart date suggestion
- Auto-generated itinerary
- Weather + holiday + air quality context
- Favorites or Saved Destinations module
- Role-based Admin Access Panel

2.6. Role-Based Access Model

The application implements a three-tier access control system with distinct permissions for different user types:

Guest User (Unauthenticated)

Guests can explore the app without creating an account but have limited functionality:

- **Search Destinations:** Full access to location-based search and place discovery
- **View Place Details:** Can view comprehensive information about destinations including:
 - Photos and ratings
 - Operating hours and contact information
 - Reviews and descriptions
 - Map previews and directions
- **Basic Exploration:** Browse categories and filter search results
- **Limitations:** Cannot create trip plans, or access profile features. All data is session-only and not persisted.

Regular User (Authenticated)



Authenticated users (via Email/Password or Google OAuth) have full access to all application features:

- **Trip Planning:** Create, save, and manage AI-generated multi-day travel itineraries
- **Favorites Management:** Bookmark destinations and sync across devices via Supabase
- **Saved Plans:** Persistent storage of all generated itineraries in Supabase **plans** table
- **Profile Management:**
 - Update personal information (name, email)
 - Change password
 - Upload and manage profile avatar (stored in Supabase Storage)
- **Cross-Device Sync:** All user data synchronized in real-time via Supabase
- **Search History:** Recent searches cached in **client_storage**
- **Offline Access:** Favorites cached locally in **favorites.json** for offline viewing

Admin (External Monitoring)

Administrators monitor and manage the system through the **Supabase Dashboard** (external to the app):

- **Database Monitoring:** View and analyze data in Supabase tables:
 - **profiles** - User account information
 - **favorites** - User-saved destinations
 - **plans** - Generated travel itineraries
- **User Management:** Monitor user registrations, authentication logs, and session activity
- **API Usage Tracking:** Monitor Google API quotas and billing through Google Cloud Console
- **Storage Management:** View and manage uploaded avatars in Supabase Storage
- **Security Policies:** Review and update Row Level Security (RLS) policies
- **No In-App Dashboard:** All administrative functions performed via Supabase web interface

Developer Role (Development Environment)

During application development, developers have access to debugging and configuration tools:



- **Debug Logging:** Application prints detailed debug information to console:
 - Authentication flow status
 - API request/response details
 - State management changes
 - Error stack traces
- **Environment Configuration:** Direct access to modify:
 - `.env` file for API keys and credentials
 - `config.py` for application settings
 - Supabase project settings
- **Testing Tools:** Ability to test with different user states (guest, authenticated)
- **Database Access:** Direct database queries via Supabase dashboard for debugging
- **Build Configuration:** Modify APK build settings in `build_apk.py` and `pyproject.toml`

2.7. Design Decisions / Trade-offs

- **Cloud-First Architecture:** Prioritizes Supabase cloud database for data persistence and authentication with minimal JSON fallback for offline scenarios.
- **Stateless AI:** The system does not retain user conversation history, generating plans on-demand to reduce storage overhead.
- **Sanitized Inputs:** All API calls use strict input validation to prevent injection attacks and minimize token wastage.

3. Specific Requirements

3.1. Functional Requirements (FR)

- FR-001: User Registration and Login: Support login via Email/password, Google Sign-In, and Guest mode.
- FR-002: User Profile Management: Users can update Personal info, Preferences, and Saved destinations.
- FR-003: AI-Based Travel Recommendations: AI suggests Destinations, Events, Activities, and Mood-based suggestions.
- FR-004: Location-Based Search: Using geolocation for Nearby POIs, Restaurants, Attractions, and Experience spots.
- FR-005: Travel Itinerary Generation: AI-generated multi-day plans based on Budget, Duration, and Preferred experiences.



- FR-006: Travel Plan Editing & Saving: CRUD operations on itineraries (Save, Edit, Delete).
- FR-007: Maps and Routing: Features include Directions, Travel time, and Distance.
- FR-008: Destination Details: Displays Photos, Ratings, Reviews, Operating hours, and Contact & address.
- FR-009: Favorites System: Users can bookmark places.
- FR-010: Third-Party API Integration: Uses Google Places, Maps, OpenWeather, OpenAQ, Calendarific, and generative LLM (Gemini).

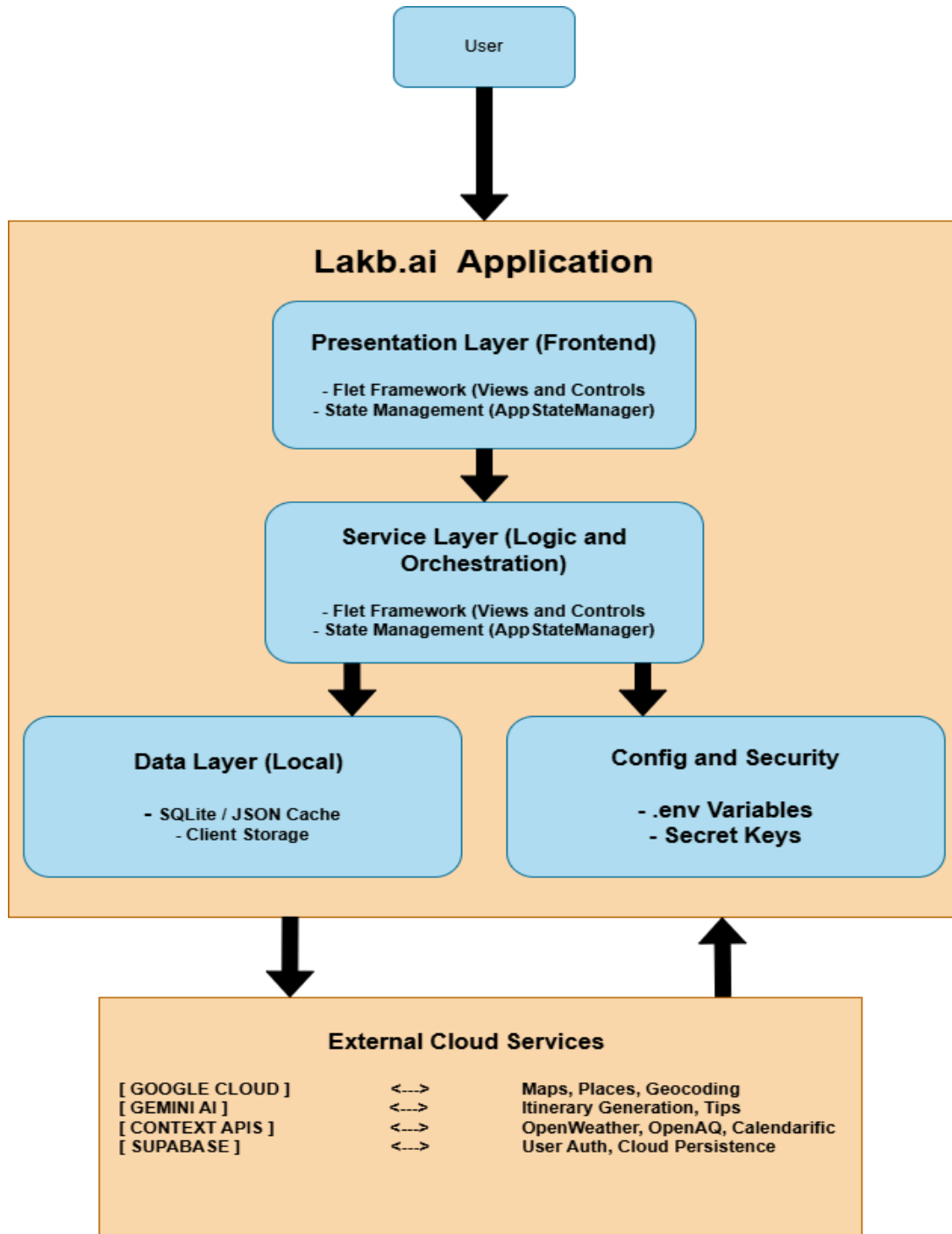
3.2. Non-Functional Requirements (NFR)

- NFR-001: Performance: Search and recommendations must load within 5 seconds under stable internet.
 - NFR-002: Usability: UI must be understandable without a tutorial; Navigation must remain consistent across pages.
 - NFR-003: Security: Encrypted API calls, Secure token storage, No API keys stored in client-side code.
 - NFR-004: Reliability: 99% uptime target; Must gracefully fail when APIs are unavailable.
 - NFR-005: Compatibility: Android 10+, iOS 15+, Browser support (Chrome, Safari, Firefox, Edge).
 - NFR-006: Scalability: Should handle growth in Users, Search queries, and API requests.
 - NFR-007: Privacy Compliance: Must abide by RA 10173 — Data Privacy Act; Location access must request user consent.
 - NFR-008: Maintainability: Modularized code, Easily replaceable AI model or API keys, Clear folder structure.
-



4. Technical Documentation

4.1. System Architecture Diagram





4.2. Threat Model & Security Controls

This section documents the potential security risks for LAKB.AI and the corresponding controls implemented to mitigate them.

Possible Threats

- API key exposure
- Unauthorized access to saved plans
- Role bypass
- Cache tampering
- Data leakage through logs
- Injection attacks
- AI misuse
- Location spoofing

Security Controls Implemented

- `.env` for API keys
- Centralized configuration using Config class
- Sanitized API inputs
- No storage of sensitive user data
- Minimal geolocation retention
- Secure DB query patterns (SQLAlchemy ORM)
- Access control checks for admin screens
- Error-safe fallbacks

5. User Manual

5.1. Installation & Setup

5.1.1. Clone the repository:

```
https://github.com/ale-xanderr/Lakb.ai
```

```
cd lakb.ai
```

5.1.2. Create and activate a Python virtual environment (Windows PowerShell):

```
python -m venv .venv
```



```
venv\Scripts\Activate.ps1
```

5.1.3. Install dependencies

```
pip install -e .
```

5.1.4. Configure environment variables

```
cp .env.example .env
```

```
# Edit .env with your API keys and credentials
```

Required environment variables:

- `SUPABASE_URL` - Your Supabase project URL
- `SUPABASE_KEY` - Supabase anonymous key
- `GOOGLE_PLACES_API_KEY` - Google Places API key
- `GOOGLE_MAPS_API_KEY` - Google Maps API key
- `GEMINI_API_KEY` - Google Gemini API key

5.2. Supabase Setup: Authentication and Database Integration

The application requires a Supabase project with authentication and database tables configured.

5.2.1. Create Supabase Project

- Go to <https://supabase.com> and create a free account
- Create a new project and note your project URL and API keys
- Copy the `SUPABASE_URL` and `SUPABASE_KEY` (anon/public key) to your `.env` file

5.2.2. Configure Authentication

Enable Authentication Providers:

- In your Supabase dashboard, navigate to **Authentication** → **Providers**
- Enable **Email** provider (enabled by default)
- Enable **Google** provider:
 - Create OAuth credentials in Google Cloud Console



- Add authorized redirect URI:
`https://<your-project-ref>.supabase.co/auth/v1/callback`
- Add the Google OAuth Client ID and Client Secret to Supabase
- Configure Site URL and Redirect URLs:
 - Site URL: `http://localhost:8550` (for development)
 - Redirect URLs: Add
`http://localhost:8550/oauth_callback` and
`lakbai://oauth_callback` (for Android deep linking)

5.2.3. Create Database Table

Run the following SQL queries in the Supabase SQL Editor (**Database** → **SQL Editor**):

Create Profiles Table:

```
-- Profiles table for extended user information
CREATE TABLE profiles (
  id UUID REFERENCES auth.users(id) PRIMARY KEY,
  email TEXT NOT NULL,
  first_name TEXT DEFAULT '',
  last_name TEXT DEFAULT '',
  avatar_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Enable Row Level Security
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;

-- Allow users to read their own profile
CREATE POLICY "Users can view own profile"
ON profiles FOR SELECT
USING (auth.uid() = id);

-- Allow users to insert their own profile
CREATE POLICY "Users can insert own profile"
ON profiles FOR INSERT
WITH CHECK (auth.uid() = id);
```



```
-- Allow users to update their own profile
CREATE POLICY "Users can update own profile"
  ON profiles FOR UPDATE
  USING (auth.uid() = id);
```

Create Favorites Table:

```
-- Favorites table for user-saved places
CREATE TABLE favorites (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT
  NULL,
  place_id TEXT NOT NULL,
  data JSONB NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  UNIQUE(user_id, place_id)
);

-- Create index for faster queries
CREATE INDEX idx_favorites_user_id ON favorites(user_id);
CREATE INDEX idx_favorites_place_id ON favorites(place_id);

-- Enable Row Level Security
ALTER TABLE favorites ENABLE ROW LEVEL SECURITY;

-- Allow users to view their own favorites
CREATE POLICY "Users can view own favorites"
  ON favorites FOR SELECT
  USING (auth.uid() = user_id);

-- Allow users to insert their own favorites
CREATE POLICY "Users can insert own favorites"
  ON favorites FOR INSERT
  WITH CHECK (auth.uid() = user_id);

-- Allow users to delete their own favorites
CREATE POLICY "Users can delete own favorites"
  ON favorites FOR DELETE
  USING (auth.uid() = user_id);
```



Create Plans Table:

```
-- Plans table for travel itineraries
CREATE TABLE plans (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT
  NULL,
  title TEXT NOT NULL,
  description TEXT,
  image_url TEXT,
  data JSONB NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Create index for faster queries
CREATE INDEX idx_plans_user_id ON plans(user_id);

-- Enable Row Level Security
ALTER TABLE plans ENABLE ROW LEVEL SECURITY;

-- Allow users to view their own plans
CREATE POLICY "Users can view own plans"
  ON plans FOR SELECT
  USING (auth.uid() = user_id);

-- Allow users to insert their own plans
CREATE POLICY "Users can insert own plans"
  ON plans FOR INSERT
  WITH CHECK (auth.uid() = user_id);

-- Allow users to update their own plans
CREATE POLICY "Users can update own plans"
  ON plans FOR UPDATE
  USING (auth.uid() = user_id);

-- Allow users to delete their own plans
CREATE POLICY "Users can delete own plans"
  ON plans FOR DELETE
  USING (auth.uid() = user_id);
```



5.2.4. Configure Storage for Profile Images

- In Supabase dashboard, go to Storage
- Create a new bucket named **avatars**
- Make the bucket public (for profile image access)
- Set up RLS policies for the storage bucket:

```
-- Allow authenticated users to upload their own avatars
CREATE POLICY "Users can upload own avatar"
ON storage.objects FOR INSERT
WITH CHECK (
    bucket_id = 'avatars'
    AND auth.uid()::text = (storage.foldername(name))[1]
);

-- Allow users to update their own avatars
CREATE POLICY "Users can update own avatar"
ON storage.objects FOR UPDATE
USING (
    bucket_id = 'avatars'
    AND auth.uid()::text = (storage.foldername(name))[1]
);

-- Allow public read access to avatars
CREATE POLICY "Public avatar access"
ON storage.objects FOR SELECT
USING (bucket_id = 'avatars');
```

5.2.5. Verify Database Setup

After running all SQL queries, verify that:

- All three tables (**profiles**, **favorites**, **plans**) exist
- Row Level Security (RLS) is enabled on all tables
- Storage bucket **avatars** is created and accessible
- Authentication providers (Email, Google) are enabled

5.3. Running the application

Development Testing (Desktop):

For quick testing during development only



Desktop:

```
flet run src\main.py
```

Web:

```
flet run src\main.py -- web -- port 8550
```

Android APK Build (Primary Platform):

```
python build_apk.py
```

5.4. Usage Guide

- Launch the app. You should see a splash screen and then the login/register screen.
- Sign in using Google or email (*if Supabase & Google OAuth is configured*). You can also continue as a guest where supported.
- Home screen features:
 - Search bar: type a place, category, or nearby search.
 - Category tabs & filters: narrow results by type (e.g., food, attractions, landmarks).
 - Destination cards: view place details, photos, and quick actions.
 - Favorites: save places to your favorites (*requires logged-in user for persistence*).
 - Plan Trip: create a date-range and parameters; generate an AI itinerary.
 - Profile / Settings: edit users details, password and save.
- Plan trip flow:
 - Open "Plan Trip" from the navigation or floating action button.
 - Select dates, preferences (budget, activity types) and request an itinerary.
 - The AI engine will propose a plan — you can save it to your profile or export/share it.
 - The generated plan trip will be available in a minute.

6. Limitations & Future Work

6.1. Limitations

- Limited offline support: the app does not provide full offline caching of place data, maps, or itineraries. Some UI state is stored in Flet's `client\_storage` (recent searches, tokens) but large datasets are not cached for offline use.



- Maps, place photos and routing rely on Google APIs (`GOOGLE\_PLACES\_API\_KEY`, `MAPS\_STATIC\_API\_KEY`) and related billing/quotas. Missing or restricted API access will result in degraded map/photo features.
- AI suggestions are powered by the configured LLM provider (the repo references `GEMINI\_API\_KEY` / generative endpoints). The current AI engine produces helpful but sometimes generic itineraries and summaries — advanced personalization and multi-pass planning are limited.
- Authentication & persistence depend on Supabase: the app integrates Supabase for OAuth (Google) and email/password flows. When Supabase credentials are not configured, authentication, profile sync, and favorites persistence will not work (the app still runs in a degraded/local-only mode).
- PKCE / OAuth caveat: the Supabase PKCE flow stores a `code\_verifier` in the running client instance. If you restart the app after initiating OAuth in the browser but before the callback completes, the code exchange may fail and require re-initiating the login flow.
- **No admin dashboard:** there is currently no built-in admin/control panel for managing destinations, users, or content from inside the app UI.
- **Limited automated tests and CI:** the repository has minimal or no automated tests or CI configured to validate features across changes.

6.2. Future Work

- **Offline-first & sync:** add local caching for search results, destination details, and saved itineraries, plus background synchronization to Supabase when connectivity returns.
- **Improved AI personalization:** refine the AI engine to use user history, saved favorites, and contextual signals (weather, local events) for more customized and non-generic itineraries.
- **Transport routing & Directions:** integrate Google Directions (or an alternative) to provide multimodal routing (walking, driving, transit) and estimated travel times between itinerary stops.
- **Real-time alerts & notifications:** add push notifications or in-app alerts for itinerary changes, weather warnings, or travel disruptions.
- **Sharing and export:** implement email/SMS sharing, calendar export (iCal), and deep links for saving/sharing itineraries.
- **Admin panel & moderation tools:** add a dedicated web admin dashboard for curating destinations, managing users, and viewing analytics and not just relying on *supabase dashboard*.



Republic of the Philippines

Camarines Sur Polytechnic Colleges

Nabua, Camarines Sur

ISO 9001:2015 Certified

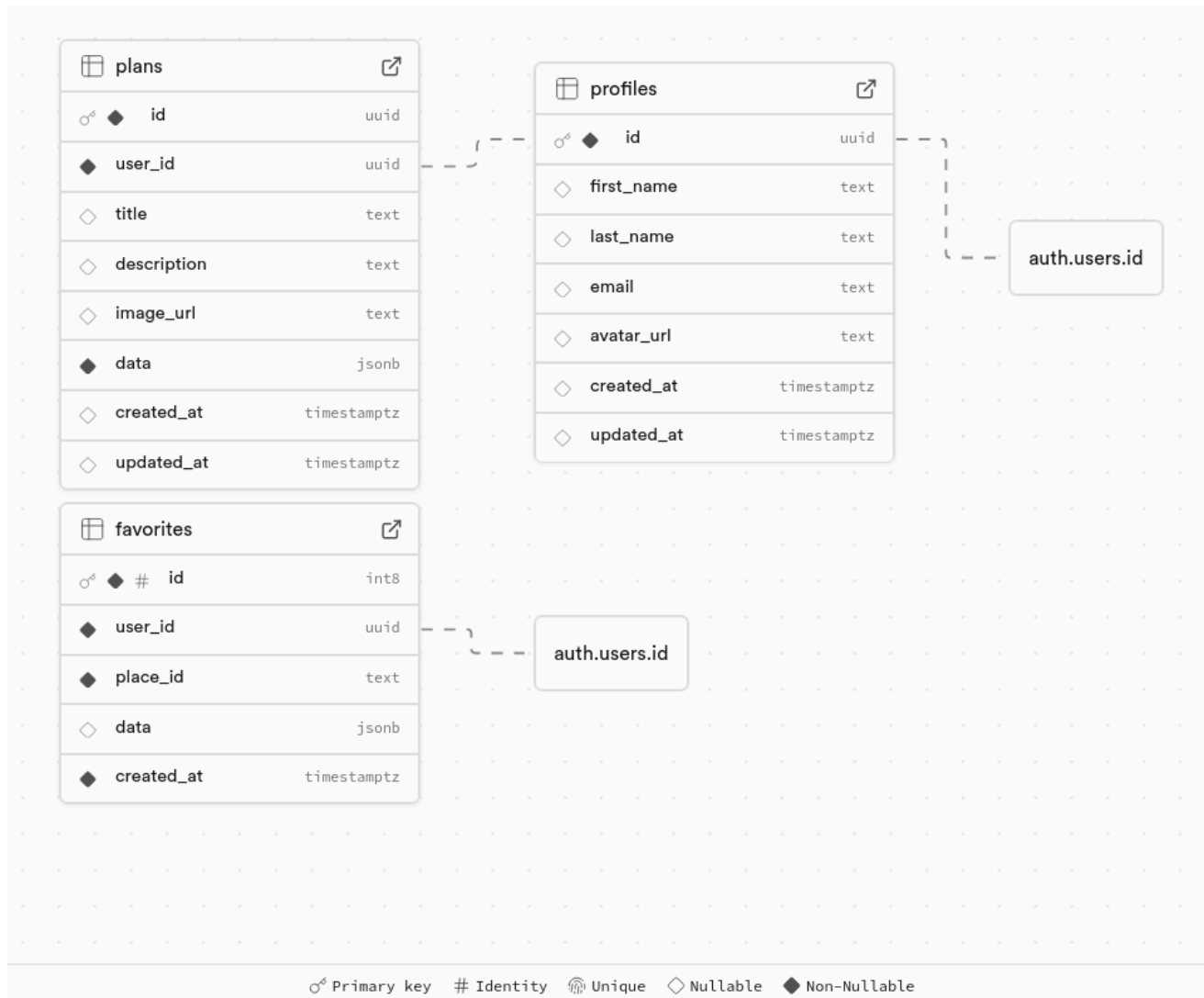
COLLEGE *of* COMPUTER STUDIES

- **Multi-device sync and offline merge:** improve how user data (favorites, plans) syncs across devices and handles conflict resolution when offline edits occur.
- **Observability & testing:** add structured logging, error reporting (Sentry or similar), unit/integration tests, and CI pipelines to maintain code quality.



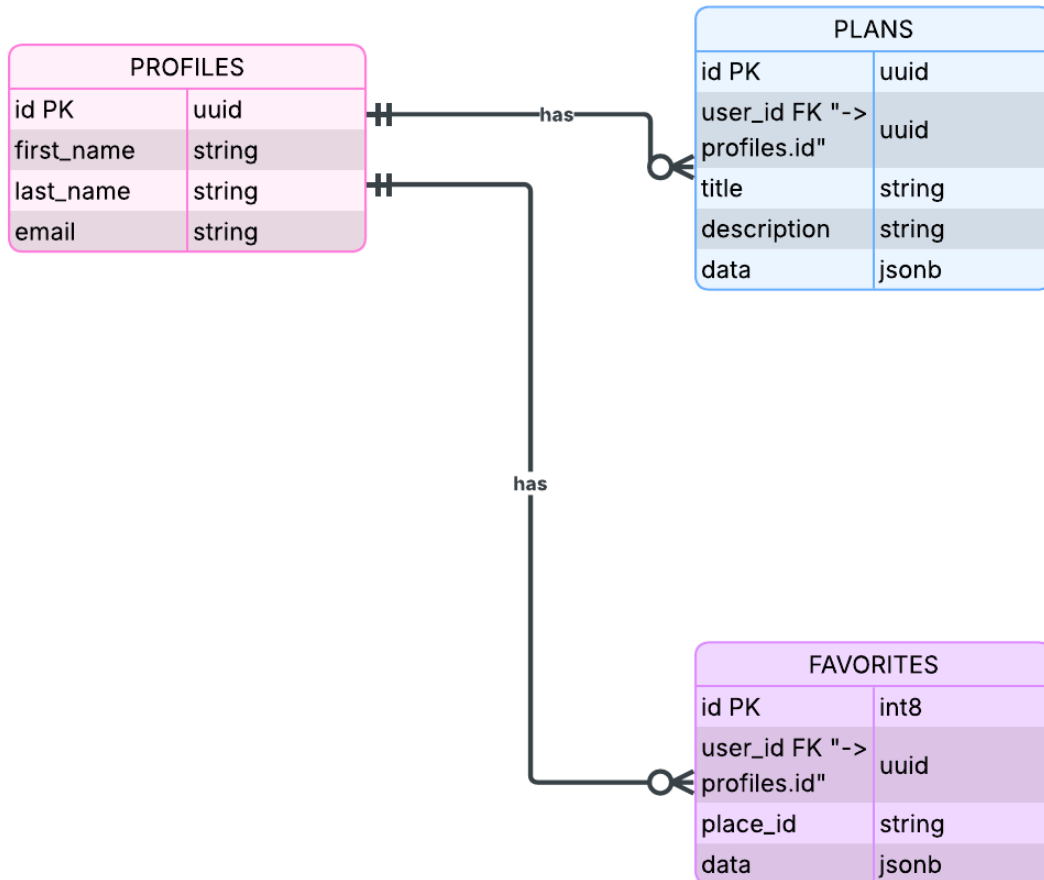
DATABASE DIAGRAM

Supabase public tables schema





Database ERD: Profiles, Plans, Favorites





LAKB.AI ERD Diagram

